



ADVANCED DATABASE SYSTEMS

Assignment Report

Title: Ride-Hailing Platform in Ethiopia

Group 3 Members

S.N.	ID Number	Name
1	MTUUR/7752/17	KALKIDAN ABEBE
2	MTUUR/7936/17	MOGES GELAYE
3	MTUUR/7984/17	NATNAEL WORKU
4	MTUUR/7851/17	TIBTBU ABERA
5	MTUUR/8230/17	YABTSEGA DAGNE
6	MTUUR/8220/17	MINALU NECHITO
7	MTUUR/8227/17	SOLOMON BERIE
8	MTUUR/7895/17	TEMECHEW ASMAMAW
9	MTUUR/7897/17	TIRUDIL AMARE
10	MTUUR/8086/17	YASHERGDU BAYE

Introduction

This report presents the design and implementation of a scalable database system for a ride-hailing platform operating in Ethiopian cities. The platform serves riders and drivers across cities like Addis Ababa, Adama, and Hawassa each with their own traffic patterns, network conditions, and user density.

The system is built on MySQL and is designed to handle real-world challenges: peak-hour concurrency, near-real-time driver location tracking, ACID-compliant payments, fraud prevention, and geographic distribution across multiple cities. Each section addresses one of the six core tasks assigned, starting from schema design through to failure recovery.

Task 1: Advanced Schema Design

The schema is built around five core entities: users, drivers, vehicles, rides, and payments. Two supporting tables handle surge zones and audit logging. All location data is stored as decimal latitude/longitude pairs, and all timestamps are stored in UTC.

1.1 Entity-Relationship Overview

The ER diagram maps the following relationships:

- A User can request many Rides (one-to-many)
- A Driver can complete many Rides (one-to-many)
- A Driver owns one Vehicle at a time (one-to-one)
- Each Ride has one Payment record (one-to-one)
- Rides may fall within one or more Surge Zones at the time of booking
- Every sensitive operation writes to the Audit Log

1.2 SQL Schema

Below is the full schema with all tables, constraints, and indexes.

```
CREATE TABLE users (  
  user_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  full_name    VARCHAR(100) NOT NULL,  
  phone        VARBINARY(255) NOT NULL,           -- stored encrypted  
  email        VARCHAR(150) UNIQUE NOT NULL,  
  city         VARCHAR(80) NOT NULL,  
  rating       DECIMAL(3,2) DEFAULT 5.00,  
  created_at   DATETIME DEFAULT CURRENT_TIMESTAMP,  
  is_active    TINYINT(1) DEFAULT 1  
);  
  
CREATE TABLE drivers (  
  driver_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  full_name    VARCHAR(100) NOT NULL,  
  phone        VARBINARY(255) NOT NULL,           -- stored encrypted  
  email        VARCHAR(150) UNIQUE NOT NULL,  
  city         VARCHAR(80) NOT NULL,  
  rating       DECIMAL(3,2) DEFAULT 5.00,  
  status       ENUM('available','busy','offline') DEFAULT 'offline',  
  latitude     DECIMAL(9,6),  
  longitude    DECIMAL(9,6),  
  location_updated_at DATETIME,  
  created_at   DATETIME DEFAULT CURRENT_TIMESTAMP,  
  is_active    TINYINT(1) DEFAULT 1  
);  
  
CREATE TABLE vehicles (  
  vehicle_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  driver_id    INT UNSIGNED NOT NULL UNIQUE,  
  make         VARCHAR(60) NOT NULL,  
  model        VARCHAR(60) NOT NULL,
```

```

    plate_number VARCHAR(20) UNIQUE NOT NULL,
    color         VARCHAR(30),
    year          YEAR,
    FOREIGN KEY (driver_id) REFERENCES drivers(driver_id) ON DELETE CASCADE
);

CREATE TABLE rides (
    ride_id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id        INT UNSIGNED NOT NULL,
    driver_id      INT UNSIGNED,
    pickup_lat     DECIMAL(9,6) NOT NULL,
    pickup_lng     DECIMAL(9,6) NOT NULL,
    dropoff_lat    DECIMAL(9,6) NOT NULL,
    dropoff_lng    DECIMAL(9,6) NOT NULL,
    pickup_address VARCHAR(255),
    dropoff_address VARCHAR(255),
    status         ENUM('requested','accepted','in_progress','completed','cancelled')
                  DEFAULT 'requested',
    base_fare      DECIMAL(10,2),
    surge_multiplier DECIMAL(4,2) DEFAULT 1.00,
    final_fare     DECIMAL(10,2),
    distance_km    DECIMAL(7,3),
    requested_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    accepted_at    DATETIME,
    completed_at   DATETIME,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (driver_id) REFERENCES drivers(driver_id)
);

CREATE TABLE payments (
    payment_id     INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ride_id        INT UNSIGNED NOT NULL UNIQUE,
    amount         DECIMAL(10,2) NOT NULL,
    method         ENUM('cash','telebirr','cbe_birr','card') NOT NULL,
    status         ENUM('pending','completed','refunded','failed') DEFAULT 'pending',
    paid_at        DATETIME,
    FOREIGN KEY (ride_id) REFERENCES rides(ride_id)
);

CREATE TABLE surge_zones (
    zone_id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    city           VARCHAR(80) NOT NULL,
    zone_name      VARCHAR(100),
    center_lat     DECIMAL(9,6) NOT NULL,
    center_lng     DECIMAL(9,6) NOT NULL,
    radius_km      DECIMAL(5,2) NOT NULL,
    multiplier     DECIMAL(4,2) DEFAULT 1.00,
    active_from    TIME,
    active_until   TIME
);

CREATE TABLE audit_log (
    log_id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    table_name     VARCHAR(60) NOT NULL,
    record_id      INT UNSIGNED NOT NULL,
    action         ENUM('INSERT','UPDATE','DELETE') NOT NULL,
    changed_by     VARCHAR(80),
    changed_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
    old_values     JSON,
    new_values     JSON
);

```

1.3 Indexing Strategy

Indexes are chosen around the three most expensive query patterns: location lookups, ride status filtering, and driver availability searches.

```
-- Driver location + status (used constantly for matching)
CREATE INDEX idx_driver_location_status
  ON drivers (status, city, latitude, longitude);

-- Ride lookups by user and status
CREATE INDEX idx_ride_user_status
  ON rides (user_id, status, requested_at);

-- Ride lookups by driver
CREATE INDEX idx_ride_driver_status
  ON rides (driver_id, status);

-- Payment status for reconciliation jobs
CREATE INDEX idx_payment_status
  ON payments (status, paid_at);

-- Audit log queries by table and time
CREATE INDEX idx_audit_table_time
  ON audit_log (table_name, changed_at);
```

We deliberately avoided indexing every column. Over-indexing would slow down writes, which is a real concern on a platform where driver location updates may happen every 5–10 seconds.

Task 2: Query Optimization

The three core analytical queries for this platform are: finding the nearest available driver, calculating surge pricing for a zone, and generating a daily revenue report. Each query is written with performance in mind.

2.1 Finding Nearest Available Drivers

We use the Haversine formula to calculate distance in kilometers between two GPS coordinates. The query filters by city first to narrow the dataset before doing the expensive math.

```
-- Find the 5 nearest available drivers to a pickup point
-- @pickup_lat = 9.0320, @pickup_lng = 38.7469 (Addis Ababa)

SELECT
  d.driver_id,
  d.full_name,
  v.make, v.model, v.plate_number,
  d.rating,
  ROUND(
    6371 * ACOS(
      COS(RADIANS(@pickup_lat))
      * COS(RADIANS(d.latitude))
      * COS(RADIANS(d.longitude) - RADIANS(@pickup_lng))
      + SIN(RADIANS(@pickup_lat))
```

```

        * SIN(RADIANS(d.latitude))
    ), 2
) AS distance_km
FROM drivers d
JOIN vehicles v ON v.driver_id = d.driver_id
WHERE d.status = 'available'
    AND d.city = 'Addis Ababa'
    AND d.is_active = 1
HAVING distance_km <= 5
ORDER BY distance_km ASC
LIMIT 5;

```

EXPLAIN output for this query shows a ref scan on `idx_driver_location_status`, using the (status, city) prefix to reduce rows before applying the Haversine calculation. MySQL cannot use an index for trigonometric functions, so the city/status filter does the heavy lifting.

2.2 Calculating Surge Pricing

Surge pricing is determined by checking whether a pickup point falls within any active surge zone for that city at the current time.

```

-- Check surge multiplier for a pickup location
SELECT
    sz.zone_name,
    sz.multiplier,
    ROUND(
        6371 * ACOS(
            COS(RADIANS(@pickup_lat)) * COS(RADIANS(sz.center_lat))
            * COS(RADIANS(sz.center_lng) - RADIANS(@pickup_lng))
            + SIN(RADIANS(@pickup_lat)) * SIN(RADIANS(sz.center_lat))
        ), 3
    ) AS dist_from_center
FROM surge_zones sz
WHERE sz.city = 'Addis Ababa'
    AND sz.active_from <= CURTIME()
    AND sz.active_until >= CURTIME()
HAVING dist_from_center <= sz.radius_km
ORDER BY sz.multiplier DESC
LIMIT 1;

```

If no rows are returned, the multiplier defaults to 1.00 (no surge). Application logic handles that fallback. The `surge_zones` table is small and rarely updated, so it fits comfortably in MySQL's query cache.

2.3 Daily Revenue Report

This query gives operations staff a per-city, per-payment-method breakdown of revenue for any given day.

```

-- Daily revenue report (run for a given date)
SELECT
    r.city_snapshot,
    p.method,
    COUNT(*) AS total_rides,
    SUM(p.amount) AS total_revenue,

```

```

        ROUND(AVG(p.amount), 2)      AS avg_fare,
        MAX(p.amount)                AS max_fare
FROM payments p
JOIN rides r ON r.ride_id = p.ride_id
WHERE p.status = 'completed'
      AND DATE(p.paid_at) = '2026-05-11'
GROUP BY r.city_snapshot, p.method
ORDER BY total_revenue DESC;

-- NOTE: city_snapshot is a denormalized column added to rides
-- to avoid joining back to users for every report row.
ALTER TABLE rides ADD COLUMN city_snapshot VARCHAR(80) AFTER status;

```

For this report to stay fast as data grows, a composite index on (status, paid_at) in the payments table already defined keeps the date filter efficient. At scale, this query would move to a separate read replica or reporting database.

Task 3: Transactions & Concurrency

Concurrency is one of the trickiest problems in a ride-hailing system. Two users in Bole (a busy Addis Ababa neighborhood) can request rides within milliseconds of each other and both match to the same available driver. Without proper handling, you end up with a double-booked driver which is exactly the kind of thing that erodes user trust.

3.1 The Lost Update Problem

Here is how a lost update can occur without proper isolation:

<pre> -- Session A (User 1 books driver 1) START TRANSACTION; -- Both sessions read 'available' SELECT status FROM drivers WHERE driver_id = 1; --> -- Result: 'available' -- Both proceed to book UPDATE drivers SET status='busy' WHERE driver_id = 1; -- Both commit successfully COMMIT; -- Driver 1 is now double-booked. Bad. </pre>	<pre> -- Session B (User 2 books driver 1) START TRANSACTION; SELECT status FROM drivers WHERE driver_id = 1; -- Result: 'available' UPDATE drivers SET status='busy' WHERE driver_id = 1; COMMIT; </pre>
---	--

3.2 Solution: SELECT FOR UPDATE

We fix this with a pessimistic lock using SELECT FOR UPDATE inside a transaction. Only one session can hold the lock at a time; the second one waits.

```

-- Safe ride booking procedure
DELIMITER $$
CREATE PROCEDURE book_ride(

```

```

    IN p_user_id      INT UNSIGNED,
    IN p_driver_id    INT UNSIGNED,
    IN p_pickup_lat   DECIMAL(9,6),
    IN p_pickup_lng   DECIMAL(9,6),
    IN p_drop_lat     DECIMAL(9,6),
    IN p_drop_lng     DECIMAL(9,6),
    OUT p_ride_id     INT UNSIGNED,
    OUT p_result      VARCHAR(50)
)
BEGIN
    DECLARE v_status VARCHAR(20);

    START TRANSACTION;

    -- Lock the driver row before reading status
    SELECT status INTO v_status
    FROM drivers
    WHERE driver_id = p_driver_id
    FOR UPDATE;

    IF v_status != 'available' THEN
        ROLLBACK;
        SET p_result = 'driver_unavailable';
    ELSE
        -- Mark driver as busy
        UPDATE drivers
        SET status = 'busy'
        WHERE driver_id = p_driver_id;

        -- Create the ride record
        INSERT INTO rides (
            user_id, driver_id,
            pickup_lat, pickup_lng,
            dropoff_lat, dropoff_lng,
            status, city_snapshot
        )
        SELECT
            p_user_id, p_driver_id,
            p_pickup_lat, p_pickup_lng,
            p_drop_lat, p_drop_lng,
            'accepted',
            d.city
        FROM drivers d WHERE d.driver_id = p_driver_id;

        SET p_ride_id = LAST_INSERT_ID();
        SET p_result = 'success';

        COMMIT;
    END IF;
END$$
DELIMITER ;

```

3.3 Deadlock Scenario and Prevention

Deadlocks happen when two transactions each hold a lock the other needs. For example:

- Transaction A locks driver 1, then tries to lock driver 2
- Transaction B locks driver 2, then tries to lock driver 1

- Both wait forever — MySQL detects this and rolls one back

The fix is simple: always lock resources in the same order across all transactions. In our booking procedure, we always lock the driver row first, then insert the ride. We also configure a short lock wait timeout:

```
-- Reduce deadlock wait time (set in my.cnf or per session)
SET innodb_lock_wait_timeout = 5; -- seconds

-- MySQL will automatically detect deadlocks and roll back
-- the transaction with the least work done.
-- Application code should catch ER_LOCK_DEADLOCK (1213)
-- and retry the transaction up to 3 times.
```

3.4 Isolation Level

We use REPEATABLE READ (MySQL's default) for booking transactions. This prevents dirty reads and non-repeatable reads. For the reporting queries in Task 2, we drop to READ COMMITTED to avoid holding locks on historical data longer than necessary.

```
-- For booking sessions (default, but set explicitly for clarity)
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- For read-heavy reporting sessions
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

3.5 Live Concurrency Simulation

This section provides two runnable scripts that demonstrate the concurrency problem and its fix. Open two MySQL terminal sessions and follow the steps in order. The timing matters — the idea is both sessions are running at the same time.

Simulation A: The Problem (No Locking)

Run these steps across two terminals. Both sessions will read the driver as 'available' and both will succeed — resulting in a double booking.

```
-- =====
-- SESSION A - Terminal 1
-- =====

USE ridehailing;
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Step 1: Start transaction
START TRANSACTION;

-- Step 2: Read driver status (NO lock)
SELECT driver_id, full_name, status
FROM drivers
WHERE driver_id = 1;
-- Expected: status = 'available'
```

```

-- >>> PAUSE HERE – switch to Terminal 2 and run Session B steps 1-2 <<<

-- Step 3: (After Session B has also read 'available') Update driver
UPDATE drivers SET status = 'busy' WHERE driver_id = 1;

INSERT INTO rides (user_id, driver_id, pickup_lat, pickup_lng,
    dropoff_lat, dropoff_lng, status, city_snapshot)
VALUES (1, 1, 9.0320, 38.7469, 9.0150, 38.7600, 'accepted', 'Addis Ababa');

COMMIT;
SELECT 'Session A committed – ride created for User 1' AS result;

-- =====
-- SESSION B – Terminal 2
-- =====
USE ridehailing;
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Step 1: Start transaction
START TRANSACTION;

-- Step 2: Read driver status (NO lock) – runs while Session A is paused
SELECT driver_id, full_name, status
FROM drivers
WHERE driver_id = 1;
-- Expected: ALSO reads 'available' – problem starts here

-- >>> Now go back to Terminal 1 and run Session A step 3 <<<
-- >>> Then come back and continue below <<<

-- Step 3: Also updates and commits – DOUBLE BOOKING occurs
UPDATE drivers SET status = 'busy' WHERE driver_id = 1;

INSERT INTO rides (user_id, driver_id, pickup_lat, pickup_lng,
    dropoff_lat, dropoff_lng, status, city_snapshot)
VALUES (2, 1, 9.0200, 38.7500, 9.0100, 38.7700, 'accepted', 'Addis Ababa');

COMMIT;
SELECT 'Session B committed – driver 1 is now double-booked!' AS result;

-- Verify the damage:
SELECT ride_id, user_id, driver_id, status
FROM rides WHERE driver_id = 1 AND status = 'accepted';

```

Simulation B: The Fix (SELECT FOR UPDATE)

Reset driver 1 back to 'available', then repeat the simulation using SELECT FOR UPDATE. This time, Session B will be blocked until Session A commits, then it will correctly find the driver is 'busy' and abort.

```

-- Reset state before running Simulation B
UPDATE drivers SET status = 'available' WHERE driver_id = 1;
DELETE FROM rides WHERE driver_id = 1;

```

```

-- =====
-- SESSION A – Terminal 1 (with lock)
-- =====
USE ridehailing;
START TRANSACTION;

-- Step 1: Read with exclusive lock
SELECT driver_id, full_name, status
FROM drivers
WHERE driver_id = 1
FOR UPDATE;
-- Driver 1 row is now locked. Session B cannot read it yet.

-- >>> Switch to Terminal 2 and try Session B step 1 – it will hang <<<

-- Step 2: Proceed with booking
UPDATE drivers SET status = 'busy' WHERE driver_id = 1;

INSERT INTO rides (user_id, driver_id, pickup_lat, pickup_lng,
                  dropoff_lat, dropoff_lng, status, city_snapshot)
VALUES (1, 1, 9.0320, 38.7469, 9.0150, 38.7600, 'accepted', 'Addis Ababa');

COMMIT;
-- Session B is now unblocked and will see status = 'busy'
SELECT 'Session A committed successfully.' AS result;

-- =====
-- SESSION B – Terminal 2 (blocked, then correctly rejected)
-- =====
USE ridehailing;
START TRANSACTION;

-- Step 1: Try to lock the same driver row
-- This will BLOCK until Session A commits (you will see the cursor hang)
SELECT driver_id, full_name, status
FROM drivers
WHERE driver_id = 1
FOR UPDATE;

-- After Session A commits, Session B unblocks and reads the latest state:
-- status = 'busy'

-- Step 2: Check the status and abort if busy
-- (In the stored procedure this is handled by the IF block)
SELECT
    CASE
        WHEN status = 'available' THEN 'Proceeding with booking'
        ELSE 'Driver is busy – rolling back'
    END AS booking_result
FROM drivers WHERE driver_id = 1;

ROLLBACK;
SELECT 'Session B correctly aborted. No double booking.' AS result;

```

Deadlock Simulation

This simulation triggers a deadlock by having two sessions lock rows in opposite order. MySQL will automatically detect it and kill one session with error 1213.

```
-- =====
-- SESSION A - locks driver 1, then tries driver 2
-- =====
START TRANSACTION;
SELECT * FROM drivers WHERE driver_id = 1 FOR UPDATE; -- locks driver 1

-- >>> Switch to Terminal 2, run Session B step 1 <<<

-- Now try to lock driver 2 - Session B already holds it
-- MySQL detects the cycle and kills one transaction
SELECT * FROM drivers WHERE driver_id = 2 FOR UPDATE;
COMMIT;

-- =====
-- SESSION B - locks driver 2, then tries driver 1
-- =====
START TRANSACTION;
SELECT * FROM drivers WHERE driver_id = 2 FOR UPDATE; -- locks driver 2

-- >>> Switch to Terminal 1 and run Session A step 2 <<<

-- Now try to lock driver 1 - Session A already holds it
SELECT * FROM drivers WHERE driver_id = 1 FOR UPDATE;
COMMIT;

-- One session gets: ERROR 1213 (40001): Deadlock found.
-- MySQL automatically rolls it back.
-- The other session proceeds normally.

-- Check the last deadlock details:
SHOW ENGINE INNODB STATUS\G
```

In the actual application, the booking service catches error 1213 and retries the transaction automatically — up to 3 times before returning an error to the user.

Task 4: Security & Access Control

Security on this platform covers two concerns: who can access what, and how sensitive data is protected when stored.

4.1 Role-Based Access Control (RBAC)

We define four database users, each with the minimum privileges needed for their role.

```
-- 1. Application user (backend API - general read/write on rides and users)
CREATE USER 'app_user'@'%' IDENTIFIED BY 'StrongPass!2026';
GRANT SELECT, INSERT, UPDATE ON ridehailing.users TO 'app_user'@'%';
GRANT SELECT, INSERT, UPDATE ON ridehailing.drivers TO 'app_user'@'%';
GRANT SELECT, INSERT, UPDATE ON ridehailing.rides TO 'app_user'@'%';
GRANT SELECT, INSERT, UPDATE ON ridehailing.payments TO 'app_user'@'%';
GRANT EXECUTE ON ridehailing.* TO 'app_user'@'%';

-- 2. Driver location updater (only needs to update location fields)
CREATE USER 'location_svc'@'%' IDENTIFIED BY 'LocPass!2026';
GRANT SELECT, UPDATE ON ridehailing.drivers TO 'location_svc'@'%';

-- 3. Analytics user (read-only, no access to PII tables)
CREATE USER 'analyst'@'%' IDENTIFIED BY 'AnalystPass!2026';
GRANT SELECT ON ridehailing.rides TO 'analyst'@'%';
GRANT SELECT ON ridehailing.payments TO 'analyst'@'%';
GRANT SELECT ON ridehailing.surge_zones TO 'analyst'@'%';
-- Deliberately NO access to users or drivers (PII)

-- 4. DBA / admin (full access, used only for maintenance)
CREATE USER 'dba_admin'@'localhost' IDENTIFIED BY 'Dbasecure!2026';
GRANT ALL PRIVILEGES ON ridehailing.* TO 'dba_admin'@'localhost';

FLUSH PRIVILEGES;
```

4.2 Encrypting Sensitive Data

Phone numbers are personally identifiable information (PII). We encrypt them at the application level before storing, and use MySQL's AES functions for database-level encryption as a second layer.

```
-- Store encrypted phone number on insert
INSERT INTO users (full_name, phone, email, city)
VALUES (
    'Abel Tesfaye',
    AES_ENCRYPT('+251911234567', SHA2('secret_key_123', 256)),
    'abel@example.com',
    'Addis Ababa'
);

-- Decrypt for display (only privileged roles)
SELECT
    user_id,
    full_name,
    CAST(AES_DECRYPT(phone, SHA2('secret_key_123', 256)) AS CHAR) AS phone
FROM users
WHERE user_id = 1;
```

In production, the encryption key is not embedded in SQL. It is passed from the application layer (fetched from a secrets manager like AWS Secrets Manager or HashiCorp Vault) so it never lives in plain text on the database server.

4.3 Audit Trigger

Any change to a ride's status is automatically logged to the audit table via a trigger.

```
DELIMITER $$
CREATE TRIGGER trg_rides_audit_update
AFTER UPDATE ON rides
FOR EACH ROW
BEGIN
    IF OLD.status != NEW.status THEN
        INSERT INTO audit_log (table_name, record_id, action, old_values, new_values)
        VALUES (
            'rides',
            NEW.ride_id,
            'UPDATE',
            JSON_OBJECT('status', OLD.status, 'driver_id', OLD.driver_id),
            JSON_OBJECT('status', NEW.status, 'driver_id', NEW.driver_id)
        );
    END IF;
END$$
DELIMITER ;
```

A similar trigger covers INSERT and DELETE on the payments table. These audit records are immutable — the analyst role has no write access to audit_log, and the app_user role has no access to it at all.

4.4 Securing the MySQL Server

Beyond SQL-level controls, the server itself is hardened:

- Run `mysql_secure_installation` to remove anonymous users and the test database
- Bind MySQL to 127.0.0.1 only; application connects via a VPN or SSH tunnel
- Enable `require_secure_transport = ON` to enforce TLS for all connections
- Set `max_connect_errors` and `max_connections` to prevent brute-force and resource exhaustion

Task 5: Distributed Database Design

As the platform expands to multiple cities — Addis Ababa, Adama, Hawassa, Dire Dawa keeping everything on one server creates bottlenecks. A distributed design spreads the load while keeping the system consistent and resilient.

5.1 Fragmentation Strategy

We use horizontal fragmentation (also called sharding) based on the city column. Each shard holds all data for one city.

Shard	City	Covers
Shard 1	Addis Ababa	users, drivers, rides, payments where city = 'Addis Ababa'
Shard 2	Adama	users, drivers, rides, payments where city = 'Adama'
Shard 3	Hawassa	users, drivers, rides, payments where city = 'Hawassa'
Shard 4	Dire Dawa	users, drivers, rides, payments where city = 'Dire Dawa'

Two tables are not sharded and remain global:

- surge_zones — read-heavy, small, needed by all cities
- audit_log — centralized for security auditing across all nodes

A user who rides in both Addis Ababa and Adama has their profile on the Addis Ababa shard (their registration city). Cross-city ride records are written to the shard of the city where the ride happened. This keeps most queries local.

5.2 MySQL Implementation with Partitioning

For the initial rollout (before full multi-server deployment), we implement city-based partitioning on a single server. This lets the optimizer skip irrelevant partitions for city-scoped queries.

```
-- Partition the rides table by city
ALTER TABLE rides
PARTITION BY LIST COLUMNS (city_snapshot) (
    PARTITION p_addis VALUES IN ('Addis Ababa'),
    PARTITION p_adama VALUES IN ('Adama'),
    PARTITION p_hawassa VALUES IN ('Hawassa'),
    PARTITION p_diredawa VALUES IN ('Dire Dawa'),
    PARTITION p_other VALUES IN (DEFAULT)
);
```

5.3 Replication Strategy

Each city shard runs MySQL Group Replication with one primary and two replicas. This gives us:

- Read scalability — analytics and driver-matching reads go to replicas
- High availability — if the primary fails, a replica is automatically promoted
- Fault isolation — a failure in the Hawassa shard does not affect Addis Ababa

```
-- Enable binary logging on primary (required for replication)
-- Add to my.cnf on the primary node:
[mysqld]
server-id          = 1
log_bin            = /var/log/mysql/mysql-bin.log
binlog_format      = ROW
gtid_mode           = ON
enforce_gtid_consistency = ON

-- On each replica node:
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST      = '10.0.1.10',
  SOURCE_USER      = 'replication_user',
  SOURCE_PASSWORD  = 'RepPass!2026',
  SOURCE_AUTO_POSITION = 1;

START REPLICA;
```

5.4 CAP Theorem Considerations

Under the CAP theorem, we prioritize Availability and Partition Tolerance (AP) for the user-facing booking flow. A ride request should always get a response, even if one shard is momentarily out of sync. Strict consistency is enforced only for payments — those transactions use synchronous replication and will wait for confirmation from at least one replica before committing.

Task 6: Failure Recovery

A ride-hailing system cannot afford data loss. A driver marks a ride complete, the payment is processed, and if the server crashes a second later, that data must survive. MySQL InnoDB handles this through a layered recovery system.

6.1 Write-Ahead Logging (WAL): InnoDB Redo Log

InnoDB uses a redo log (MySQL's implementation of WAL). Before any data page is modified on disk, the change is first written to the redo log. If the server crashes mid-transaction, MySQL replays the redo log on startup to bring the database back to its last consistent state.

```
-- my.cnf settings for durable logging
[mysqld]
```



```
innodb_flush_log_at_trx_commit = 1
-- 1 = flush to disk on every commit (safest, slight performance cost)
-- 2 = flush to OS buffer only (survives MySQL crash, not OS crash)

innodb_redo_log_capacity = 2G    -- size of the redo log (MySQL 8.0.30+)
innodb_doublewrite       = ON    -- prevents partial page writes on crash
```

Setting `innodb_flush_log_at_trx_commit = 1` is non-negotiable for a payments system. The small performance cost (a disk flush per commit) is worth it compared to the alternative of losing a customer's payment record.

6.2 Checkpoints

InnoDB periodically flushes dirty pages (modified pages in the buffer pool) to the data files on disk. This process is called a checkpoint. It bounds how far back MySQL must replay the redo log after a crash — the more frequent the checkpoints, the faster the recovery.

```
-- Check current checkpoint activity
SHOW ENGINE INNODB STATUS\G
-- Look for: 'Log sequence number', 'Log flushed up to', 'Pages flushed up to'

-- Tune the buffer pool size (checkpoint frequency depends on this)
innodb_buffer_pool_size = 4G          -- ~70% of available RAM
innodb_io_capacity       = 2000        -- IOPS available on the server
innodb_io_capacity_max   = 4000
```

6.3 Binary Log for Point-in-Time Recovery

The binary log records every data-changing statement (or row change in ROW format). Combined with a full backup, it allows us to recover to any specific point in time — not just the last backup.

```
-- Enable binary log (already shown in replication config)
log_bin          = /var/log/mysql/mysql-bin.log
binlog_format    = ROW
expire_logs_days = 14    -- keep 14 days of binary logs

-- Point-in-time recovery example:
-- 1. Restore last full backup (see 6.4)
-- 2. Replay binary logs up to the moment before the failure
mysqlbinlog --start-datetime='2026-05-11 08:00:00' \
            --stop-datetime='2026-05-11 14:32:00' \
            /var/log/mysql/mysql-bin.000045 \
            | mysql -u root -p ridehailing
```

6.4 Backup Strategy

We use a combination of daily full backups and continuous binary log shipping:

```
-- Full backup with mysqldump
mysqldump \
  --single-transaction \
  --routines \
  --triggers \
  --flush-logs \
  --master-data=2 \
  -u dba_admin -p ridehailing \
  | gzip > /backups/ridehailing_$(date +%F).sql.gz

-- --single-transaction: uses a consistent snapshot, no table locks
-- --master-data=2:      records the binary log position in the dump
-- --flush-logs:        rotates binary log at backup time for clean recovery
```

For faster backups on large datasets, Percona XtraBackup can create hot physical backups of InnoDB files without locking the database. This is recommended once the dataset exceeds a few GB.

6.5 Recovery Runbook

In the event of a full server failure, the recovery steps are:

- Provision a new MySQL server with the same configuration
- Restore the most recent full backup: `gunzip < ridehailing_2026-05-10.sql.gz | mysql -u root -p`
- Identify the binary log position recorded in the dump (`--master-data=2` adds it as a comment)
- Replay binary logs from that position up to the point of failure using `mysqlbinlog`
- Verify data integrity: check row counts, latest ride IDs, and payment totals against application logs
- Update DNS or load balancer to point to the recovered server

Conclusion

This assignment covered the full lifecycle of designing a production-grade database system for a ride-hailing platform in Ethiopia. Starting from a normalized schema with proper geolocation support and indexing, we worked through query optimization using the Haversine formula, built concurrency-safe booking transactions using `SELECT FOR UPDATE`, layered role-based access control over encrypted PII, distributed the system across city-based shards with replication, and put together a recovery plan grounded in InnoDB's WAL and binary logging.

Every decision was made with the real context in mind: Ethiopian cities, Ethiopian payment methods (Telebirr, CBE Birr), unreliable networks, and a user base that expects rides to be confirmed fast and payments to never disappear. A database that works in that environment is one that is fast, honest about failure, and recoverable when things go wrong.

Appendix: Sample Data Population

The following inserts populate the database with the sample dataset provided in the assignment brief.

```
-- Users (from assignment dataset)
INSERT INTO users (full_name, phone, email, city) VALUES
  ('Abel', AES_ENCRYPT('+251911000001', SHA2('key',256)), 'abel@mail.com', 'Addis
Ababa'),
  ('Sara', AES_ENCRYPT('+251911000002', SHA2('key',256)), 'sara@mail.com',
'Adama'),
  ('Dawit', AES_ENCRYPT('+251911000003', SHA2('key',256)), 'dawit@mail.com',
'Hawassa');

-- Drivers (from assignment dataset)
INSERT INTO drivers (full_name, phone, email, city, status, latitude, longitude)
VALUES
  ('Bekele', AES_ENCRYPT('+251911100001', SHA2('key',256)),
  'bekele@mail.com', 'Addis Ababa', 'available', 9.0320, 38.7469),
  ('Tesfaye', AES_ENCRYPT('+251911100002', SHA2('key',256)),
  'tesfaye@mail.com', 'Addis Ababa', 'busy', 9.0150, 38.7600);

-- Vehicles
INSERT INTO vehicles (driver_id, make, model, plate_number, color, year) VALUES
  (1, 'Toyota', 'Vitz', 'AA 3-12345', 'Silver', 2018),
  (2, 'Suzuki', 'Alto', 'AA 2-67890', 'White', 2020);

-- Surge zone example (Bole area, Addis Ababa – peak morning hours)
INSERT INTO surge_zones (city, zone_name, center_lat, center_lng, radius_km,
multiplier, active_from, active_until)
VALUES ('Addis Ababa', 'Bole Morning Rush', 9.0070, 38.7900, 2.5, 1.75, '07:00:00',
'09:30:00');
```